

Cambridge Telematic Integration

Project: Cambridge Telematics (CMT) SDK & Datahub Integration Design	Date: March 19 th 2026	
Project Description: IT Curves driver terminal will submit the period data to Cambridge Telematics and also record in IT Curves dispatch platform for reconciliation purposes	3rd Party Integration: Integrating the IT Curves dispatch platform with Cambridge Telematics (CMT) using the CMT mobile SDK and DataHub backend APIs	
Change Requested By : SilverRide		
Changes Details: SilverRide is currently using Fairmatic for insurance purposes and reporting driver and vehicle driving behavior and period report. SilverRide is now moving to another insurance provider which uses Cambridge Telematics and for that purpose, integration into the driver’s device needs to be replaced and backend logic also need to be updated along with the new SDK.		
Motivation for Changes: The new insurance carrier requires this integration. In future Silver Ride envisions that they would like to develop not only the period reporting but also driving behavior analytics which includes hard breaking, speeding, harsh driving, sharp turns etc. into their own driver device. Also, Silver Ride wish to expand it to have their own dashboard to keep all this information and use for insurance reporting purposes.		
Cost Proposal Estimate for	Reference to work breakdown and labor hours	
1. Client configuration, user/vehicle provisioning, association flow, auth_code endpoint, logging, testing	1.5 Man Week	7 days
2. SDK setup, auth_code retrieval, permissions, trip control, state management.	2 Man Week	10 days
3. System Integration and Verification Tests (SIV / Development)	½ Man Week	2 days
Total	2 Man Week	
Expected Delivery Date: 3 Weeks from approval		
Terms & Payment Milestones		
50% of the cost of Change Order will be paid upon approval 50% at completion of test and delivery		

Work Breakdown	Hours	Labor Classification	Standard Hourly Rate	Discounted Hourly Rate	Discounted Total
1. Set up the system configuration on backend 2. Synchronizing driver and vehicle profile with Cambridge system 3. Handshake API to provide user ID to driver device. 4. Token re-generation if token expires. 5. Re-submitting driver and vehicle association	56	Product Line Specialist + Development	\$140	\$60.00	\$3,360.00
1. Initializing the driver user credential and registering communication with Cambridge API 2. Updating all period reporting classes with new logic and new period data 3. Update backend synchronization for parallel period reporting. 4. Handle submission failure and reattempt mechanism 5. Requesting new communication token upon token expiration	80	Development	\$140	\$60.00	\$4,800.00
Project Gross Total	104			\$60.00	\$8,160.00
Existing Client Discount		50%			\$4,080.00
Total Project Cost Final					\$4,080.00

Cambridge Telematics (CMT) SDK & DataHub Integration Design

Corrected architecture and integration document

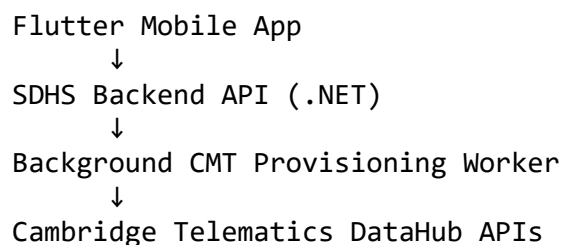
1. Overview

This document describes the design for integrating the SDHS platform with Cambridge Telematics (CMT) using the CMT mobile SDK and DataHub backend APIs.

The goal is to provision drivers and vehicles in CMT, obtain an authentication code (auth_code) for the mobile SDK, initialize tracking in the Flutter mobile app, and support the downstream lifecycle for device registration and trip collection.

2. High-Level Architecture

System flow:



The backend acts as a secure gateway between the mobile app and the Cambridge Telematics platform. The mobile application should not directly call CMT DataHub provisioning APIs.

3. Design Principles

- Do not block user login on external telematics API calls.
- Provision CMT resources asynchronously after successful login.
- Persist CMT identifiers so the same user and vehicle are not recreated.
- Return auth_code to the mobile app only through the SDHS backend.
- Treat CMT provisioning and SDK initialization as separate phases.
- Support re-provisioning, association updates, and recovery from partial failures.

4. Login and Asynchronous Provisioning Flow

1. Driver logs in through the SDHS API.
2. SDHS validates credentials and immediately returns login success.
3. A background worker is triggered for Cambridge Telematics provisioning.
4. The worker checks whether the driver already exists in CMT.
5. If missing, the worker creates or updates the driver record.

6. The worker checks whether the assigned vehicle exists in CMT.
7. If missing, the worker creates or updates the vehicle record.
8. The worker ensures user-vehicle association is correct.
9. The worker requests an auth_code from CMT.
10. The backend stores the resulting CMT identifiers and auth_code.
11. The mobile app later retrieves the auth_code from an SDHS endpoint and initializes the SDK.

5. Recommended Database Schema

Recommended backend persistence table:

```
driver_id  
cmt_user_id  
cmt_vehicle_id  
auth_code  
device_id  
provisioning_status  
last_error  
created_at  
updated_at
```

Notes:

- cmt_user_id and cmt_vehicle_id should store the CMT short identifiers returned by the backend platform.
- auth_code should be treated as a credential-like value and refreshed or replaced as required by the integration lifecycle.

6. Backend Endpoints in SDHS

Suggested internal endpoints:

```
POST /api/auth/login  
POST /api/telematics/provision  
GET /api/telematics/auth-code  
GET /api/telematics/status
```

The mobile app should call SDHS endpoints only. The SDHS backend then communicates with Cambridge Telematics DataHub.

7. Example Auth Code Retrieval Flow

1. Mobile app calls login endpoint.
2. Backend responds immediately with login success.
3. Provisioning starts in background.
4. Mobile app waits briefly or polls provisioning status.
5. Mobile app calls GET /api/telematics/auth-code.

6. Backend returns either processing or ready status.

Example JSON responses:

```
{  
  "status": "processing"  
}  
  
{  
  "status": "ready",  
  "authCode": "abc123xyz"  
}
```

8. CMT DataHub API Usage

The backend design relies on the following DataHub-style operations referenced in the earlier document. They should be treated as CMT DataHub endpoints in the implementation and documentation.

```
POST /station/v4/update_users_and_vehicles  
POST /station/v4/get_users_and_vehicles  
POST /station/v4/delete_users_and_vehicles  
POST /station/v4/request_auth_code
```

These operations support user creation/update, vehicle creation/update, association lookup, deletion, and auth_code generation.

9. User-Vehicle Association Guidance

Vehicle-driver relationships should be managed through the CMT update flow rather than by hard-coding assumptions in the mobile layer.

- Use update operations to maintain current user and vehicle data.
- When a driver changes vehicles, first evaluate whether association update is sufficient.
- If the integration or tenant rules require recreation, delete and recreate the vehicle cleanly.
- Always keep backend records synchronized with the latest CMT identifiers.

10. Device Registration and SDK Lifecycle

1. User exists in CMT.
2. Vehicle exists in CMT.
3. User is associated with vehicle.
4. Backend requests auth_code.
5. Mobile app receives auth_code from SDHS.
6. Flutter app initializes Cambridge Telematics SDK.
7. SDK performs device registration.

8. Trip tracking and event collection begin.
9. Trip, event, and scoring data become available in CMT backend systems.

11. Mobile SDK Initialization Notes

- Initialize the SDK only after `auth_code` is available.
- Avoid storing raw long-lived credentials in the mobile app.
- Handle permission prompts for motion, location, notifications, and battery optimization rules as required by the platform.
- Implement retry logic when `auth_code` is still processing.
- Surface provisioning status to the user only when operationally useful.

12. Data Retrieval Scope Beyond Provisioning

The earlier draft focused mainly on provisioning. A complete Cambridge Telematics integration normally also considers retrieval and operational use of downstream telematics data.

Typical downstream areas include:

- Trips
- Driving events
- Scores
- Behavior summaries
- Operational reporting and support workflows

These data retrieval use cases are separate from the login/provisioning flow and should be designed as independent backend services or scheduled jobs where needed.

13. Error Handling and Operational Recommendations

- Make provisioning idempotent.
- Persist partial progress after each successful CMT step.
- Store failure reason in `last_error` for troubleshooting.
- Retry transient HTTP and timeout failures with bounded backoff.
- Do not recreate users or vehicles blindly on every login.
- Log correlation IDs for SDHS request, worker execution, and CMT API call chain.
- Add dashboard visibility for provisioning success, failure, and average completion time.

14. Suggested Status Values

Status	Meaning
<code>not_started</code>	No provisioning has started yet

Status	Meaning
processing	Background provisioning is in progress
ready	auth_code is available and SDK can initialize
failed	Provisioning failed and requires retry or investigation
disabled	Telematics not enabled for this driver or vehicle

15. Example End-to-End Sequence

1. Driver logs in to SDHS
2. SDHS returns success immediately
3. SDHS worker provisions user and vehicle in CMT
4. SDHS requests auth_code from CMT
5. SDHS stores cmt_user_id, cmt_vehicle_id, and auth_code
6. Mobile app polls SDHS auth-code endpoint
7. SDHS returns auth_code
8. Flutter app initializes CMT SDK
9. SDK registers device and starts telemetry

16. Final Recommendation

The integration design is compatible with Cambridge Telematics, using auth_code for mobile SDK initialization, clear user-vehicle association handling, and coverage of the device registration lifecycle and downstream telematics scope.

This document is suitable as a CMT-focused backend and mobile integration design draft.